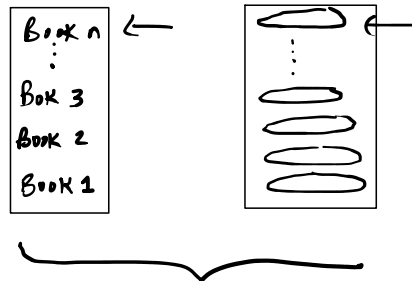


LIFO: Last in
first out

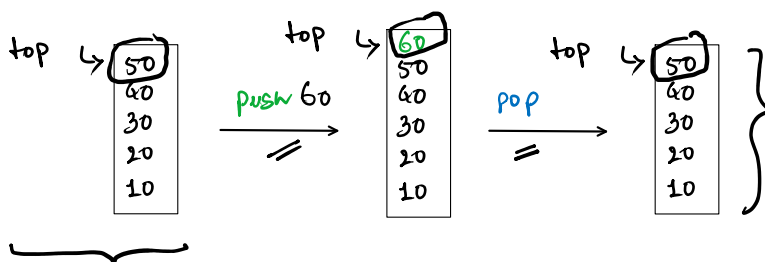
Stack

⇒ It is a "pile of objects"

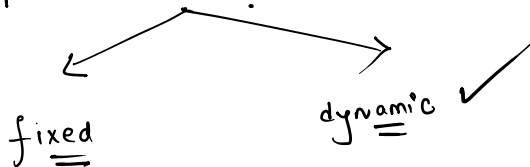
- It is a linear data-structure that supports insertion (push) and deletion (pop) from the same end (top)



Stack operates according to the LIFO principle.

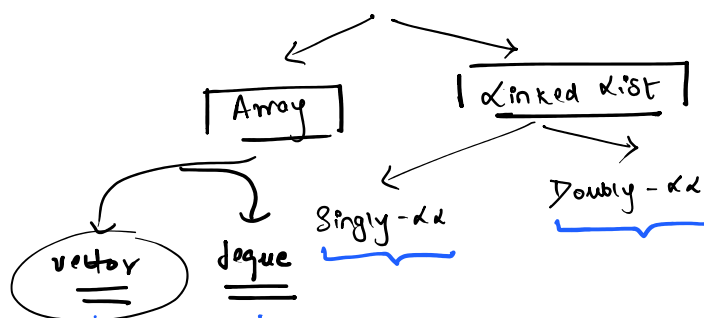


How to implement Stack?



How to represent Stack?

Container Adaptor

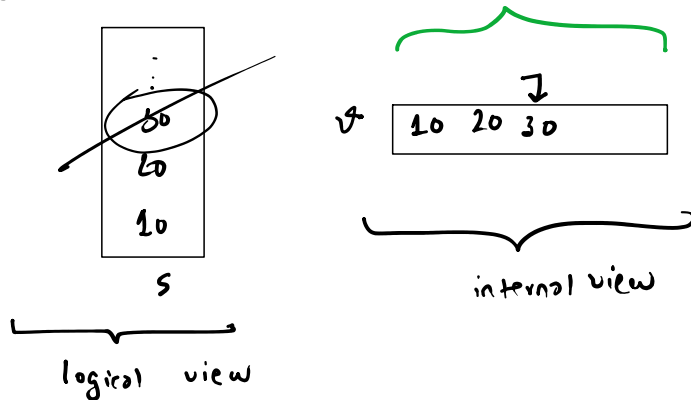


Stack Operations

1. push(data)
2. pop() ←

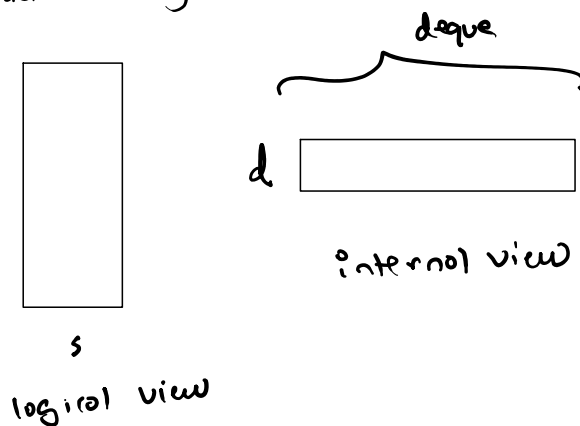
1. $\text{pop}()$ ←
2. $\text{top}()$ ←
3. $\text{size}()$ ←
4. $\text{empty}()$ ←

Dynamic Stack using vector



Logical view operations: $s.\text{push}(10)$ $s.\text{push}(20)$ $s.\text{push}(30)$ $\vdots$ $s.\text{top}()$ $s.\text{pop}()$ $s.\text{size}()$ $s.\text{empty}()$	Internal view operations: $v.\text{push-back}(10)$ $v.\text{push-back}(20)$ $v.\text{push-back}(30)$ $\vdots$ $v[n-1] / v.\text{back}()$ $v.\text{pop-back}()$ $v.\text{size}()$ $v.\text{empty}()$
--	---

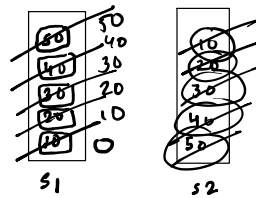
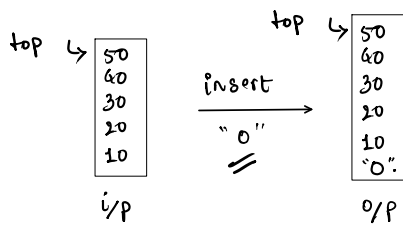
Dynamic Stack using deque



Logical view operations: $s.\text{push}(val)$ $s.\text{pop}()$ $s.\text{size}()$ $s.\text{empty}()$	Internal view operations: $d.\text{push-back}(val)$ $d.\text{pop-back}()$ $d.\text{push-front}(val)$ $d.\text{pop-front}()$ $d.\text{size}()$ $d.\text{empty}()$
---	--

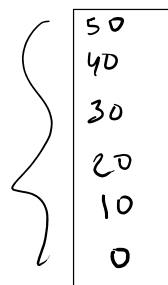
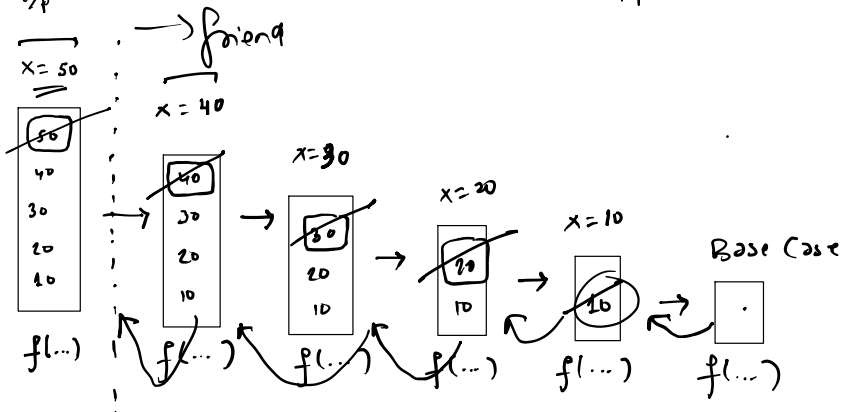
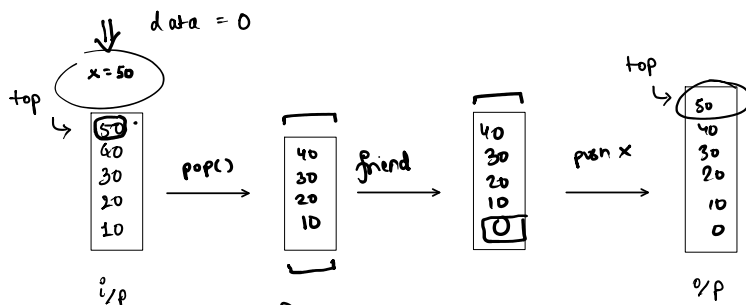
s.push(val)	→ d.push()	/	d.pop-front()
s.pop()	→ d.pop-back()		d[0] or d.front()
s.top()	→ d.back() / d[n-1]		
s.size()	→ d.size()		
s.empty()	→ d.empty()		

↳ Insert at Bottom of a Stack



space: $O(n)$
due to s2

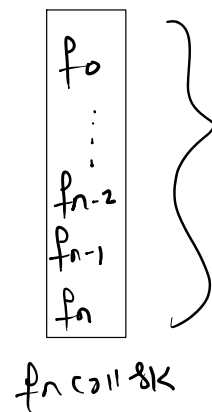
1. $s1 \rightarrow s2$ $\Rightarrow 3n$
 2. $s1.push(0)$ $\Rightarrow 1$
 3. $s2 \rightarrow s1$ $\Rightarrow 3n$
- time $\Rightarrow (6n+1) \cdot \text{worst}$
 $\therefore O(n)$



$(3n+1) \cdot \text{const}$

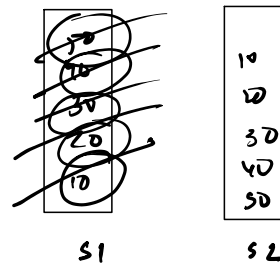
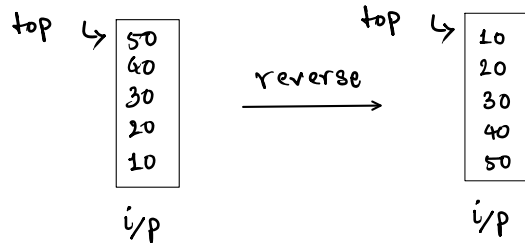
time: $O(n)$

space: $O(n)$ due to fn
(call stack)



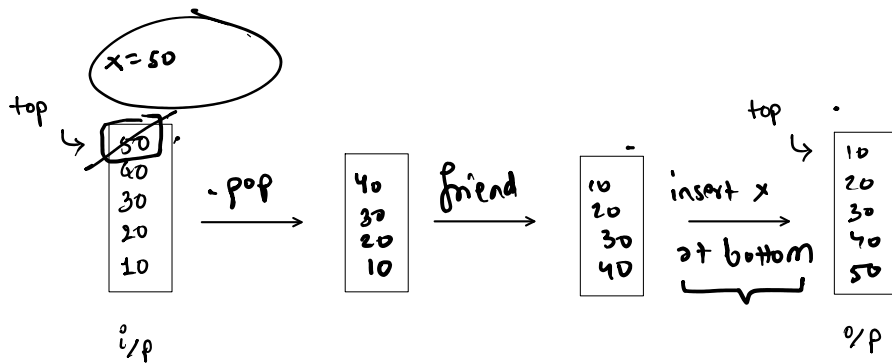
at max
we've
 n stack
frames
each stack
frame needs
const space
 $\therefore O(n)$
space

↳ Reverse Stack



1. $S1 \rightarrow S2$
2. ret S2

: $3n$ ops. const
 time: $O(n)$
 space: $O(n)$ due to S2



Valid Parentheses

Given a string **S** containing the characters '(', ')', '{', '}', '[', and ']', design an algorithm to check if the input string is valid. An input string is valid if :

- a. Open brackets must be closed by the same type of brackets.
- b. Open brackets must be closed in the correct order.
- c. Every close bracket has a corresponding open bracket of the same type.

Example :

eg:1

(	{	[	]	}	)
---	---	---	---	---	---

 → true

eg:2

(	)	[	]	{	}
---	---	---	---	---	---

 → true

eg:3

(	[	{	]	}	)
---	---	---	---	---	---

 → false

=> ({ [] })

Diagram illustrating the stack-based algorithm for the first example (eg:1):

Input string: ({ [] })

Stack operations: Push '(', Push '{', Push '[', Pop ']', Pop '{', Pop '('.

Stack state: []

Diagram illustrating the stack-based algorithm for the second example (eg:2):

Input string: ({ } [])

Stack operations: Push '(', Push '{', Pop '}', Pop '[', Pop '('.

Stack state: []

Diagram illustrating the stack-based algorithm for the third example (eg:3):

Input string: (()) ((

Stack operations: Push '(', Push '(', Pop ')', Pop '(', Push '(', Push '('.

Stack state: [((

Diagram illustrating the stack-based algorithm for the fourth example (eg:4):

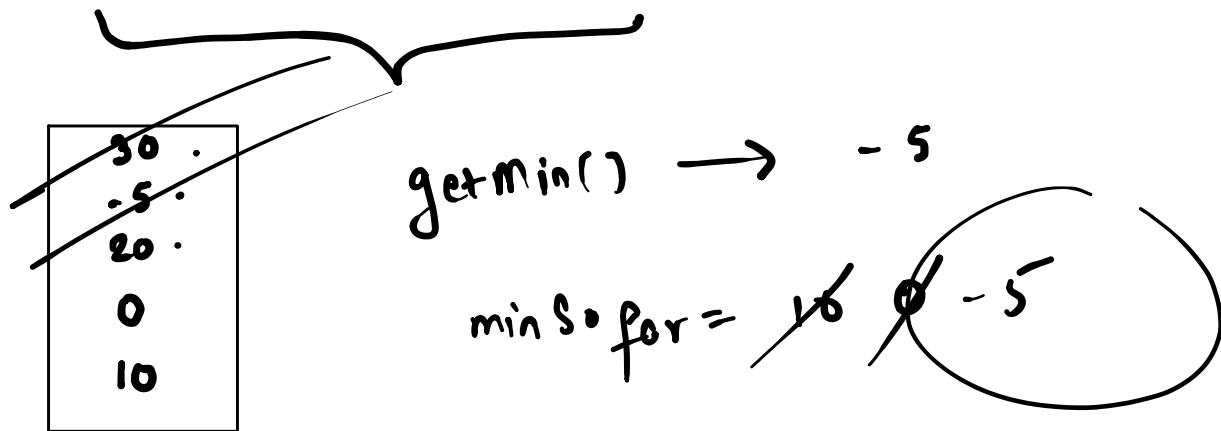
Input string: { (((

Stack operations: Push '{', Push '(', Push '(', Push '('.

Stack state: [{ (((

Min Stack

Design a stack that supports **push**, **pop**, **top**, and retrieving the **minimum** element in constant time.



Asteroid Collision

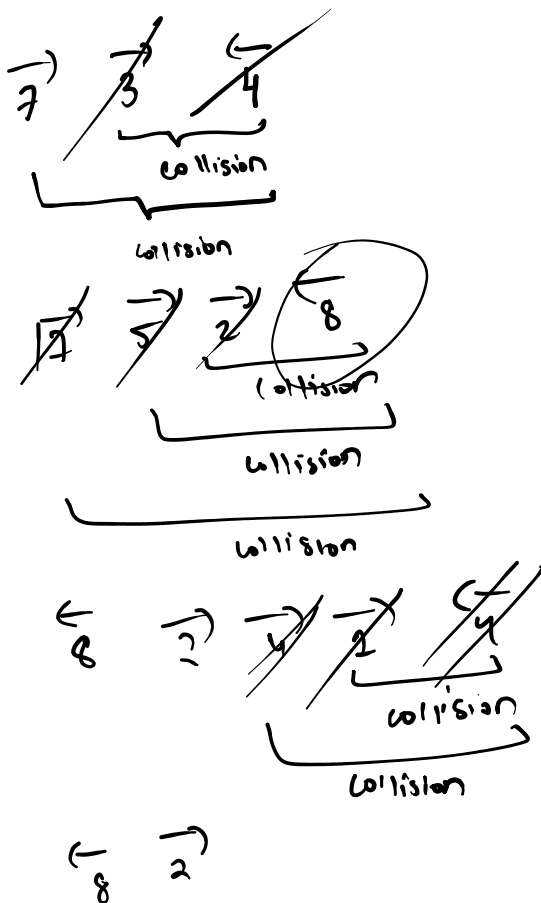
Given an integer array **asteroids** representing asteroids in a row. For each asteroid, the **absolute** value represents its **size**, and the **sign** represents its **direction** (positive meaning right, negative meaning left).

Assuming all the asteroids move at the **same speed**, find out the state of the asteroids after all the collisions such that when two asteroids meet or collide, the smaller one will explode and if both of the asteroids are of the same size then both of them will explode.

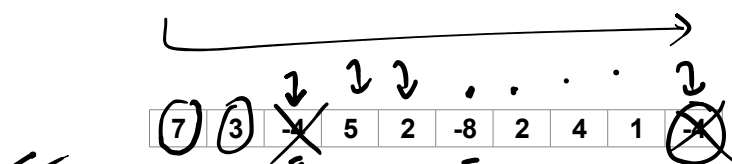
Also, two asteroids moving in the same direction will never meet.

Example :

Input	7	3	-4	5	2	-8	2	4	1	-4
Output	-8	2								

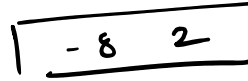
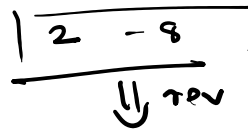
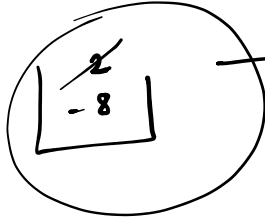
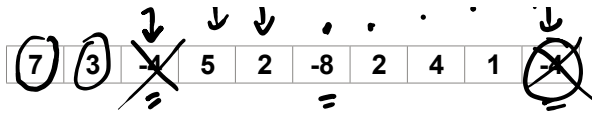
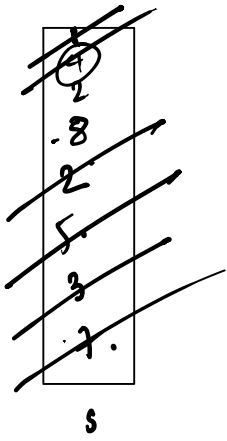


$[-8, 2]$



$\begin{matrix} + & + \\ - & - \\ - & + \end{matrix} \left. \vphantom{\begin{matrix} + & + \\ - & - \\ - & + \end{matrix}} \right\} \text{no collision}$

$\begin{matrix} + & - \end{matrix}$



[HW] Iterative Tower of Hanoi

Tower of Hanoi

Given a **three towers** and **N** disks of varying sizes that can **slide** onto any of the towers. Initially, all the **N** disks are placed in the decreasing order of their sizes on the **1st** tower (**source**) i.e. the **smallest** disk at its **top** and the largest disk at its bottom. Design an **iterative** algorithm for generating the steps required to move the **N** disks from the **source** tower to the **3rd** tower (**destination**) using the **2nd** tower (**helper**) such that at any point of time

- you can move a **one** disk from top of one tower to the top of another.
- you cannot place a **larger** disk on top of **smaller** disk

Example

Input : N = 3

Homework

<https://leetcode.com/problems/decode-string/>

<https://leetcode.com/problems/evaluate-reverse-polish-notation/>

<https://leetcode.com/problems/basic-calculator-ii/>

<https://leetcode.com/problems/basic-calculator/>

<https://leetcode.com/problems/longest-valid-parentheses/>